CO3_AT1

Analysis of Quick Sort Partition Efficiency Using Comparisons and Swaps

---

1. Aim

To implement the Quick Sort algorithm in C and analyze the efficiency of its partitioning process by counting the number of **comparisons and swaps** for different types of datasets. The experiment also aims to study how partition quality affects the overall performance of Quick Sort and justify suitable optimization strategies for efficient partitioning.

---

2. Problem Statement

Quick Sort is an efficient divide-and-conquer sorting algorithm whose performance depends heavily on the quality of the partition operation. A good partition divides the array into nearly equal-sized subarrays, while a poor partition produces highly unbalanced subarrays.

The objective of this experiment is to:

- Implement Quick Sort using partitioning.
- Count the number of comparisons performed.
- Count the number of swaps performed.
- Test the algorithm using different datasets.
- Compare the partition efficiency for sorted, reverse-sorted, random, and duplicate-value datasets.
- Analyze how pivot selection affects Quick Sort performance.
- Suggest optimization strategies for improving partition quality.

---

3. Objectives

1. To understand the partitioning process of Quick Sort.
2. To implement Quick Sort using the **Lomuto partition scheme**.
3. To measure comparisons and swaps during partitioning.
4. To evaluate Quick Sort on different input patterns.
5. To identify best-case, average-case, and worst-case behavior.
6. To understand the relationship between partition quality and recursion depth.

7. To investigate optimized pivot selection using randomized pivot selection.
8. To validate the experimental results with theoretical complexity.

---

4. Conceptual Background

4.1 Quick Sort

Quick Sort is a **divide-and-conquer sorting algorithm**.

The basic procedure is:

1. Select a pivot element.
2. Partition the array around the pivot.
3. Elements smaller than or equal to the pivot are placed on one side.
4. Larger elements are placed on the other side.
5. Recursively apply Quick Sort to both partitions.

The general recurrence is:

$T(n)=T(k)+T(n-k-1)+O(n)$

where k represents the number of elements on one side of the pivot.

4.2 Partition Quality

Partition quality determines how evenly the input array is divided.

Good partition

Example:

[1 2 3 4 5 6 7 8 9]

↑

Pivot

If the pivot divides the array approximately into:

4 elements | Pivot | 4 elements

the recursion is balanced.

This produces approximately:

performance.

Poor partition

Example:

[1 2 3 4 5 6 7 8 9]

↑

Pivot

The partition becomes:

8 elements | Pivot | 0 elements

This creates an unbalanced recursion tree and can result in: $O(n2)$

time complexity.

---

5. Partitioning Technique Used

The experiment uses the **Lomuto partition scheme**.

The last element is initially selected as the pivot.

For each element:

if array[j] <= pivot

the element is moved toward the left partition.

Finally, the pivot is placed in its correct position.

Partition structure

```
+------------------+-------+------------------+
| <= Pivot         | Pivot | > Pivot          |
+------------------+-------+------------------+
```

---

6. Experimental Design

To understand the effect of input characteristics, four different datasets are considered.

Dataset	**Description**
Sorted	Elements already arranged in ascending order
Reverse Sorted	Elements arranged in descending order

Dataset	Description
Random	Elements arranged randomly
Duplicates	Dataset containing repeated values

For each dataset, the following are recorded:

- Number of comparisons
- Number of swaps
- Final sorted array
- Partition behavior

Two pivot strategies are considered:

Strategy 1 – Last Element Pivot

The last element is always selected as the pivot.

Strategy 2 – Randomized Pivot

A random element is selected as the pivot and moved to the last position before partitioning.

The randomized approach is included to demonstrate an optimization strategy for avoiding consistently poor partitions.

---

7. Algorithm

Quick Sort Algorithm

QUICKSORT(array, low, high)

1. If low < high:
2. Select a pivot.
3. Partition the array.
4. Obtain pivot position.
5. Apply QUICKSORT to left partition.
6. Apply QUICKSORT to right partition.

7. Stop.

Lomuto Partition Algorithm

PARTITION(array, low, high)

1. Select array[high] as pivot.
2. Set i = low - 1.
3. For j = low to high - 1:
4. Compare array[j] with pivot.
5. If array[j] <= pivot:
6. Increment i.
7. Swap array[i] and array[j].
8. Swap array[i+1] and array[high].
9. Return i+1.

Source Code:

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define SIZE 10

long comparisons;
long swaps;

void swap(int *a, int *b)
{
    int temp;

    if (a != b)
    {
        temp = *a;
        *a = *b;
        *b = temp;
        swaps++;
    }
```

```
int partitionLast(int arr[], int low, int high)
{
    int pivot;
    int i;
    int j;

    pivot = arr[high];
    i = low - 1;

    for (j = low; j < high; j++)
    {
        comparisons++;

        if (arr[j] <= pivot)
        {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
```

```
    swap(&arr[i + 1], &arr[high]);

    return i + 1;
}

void quickSortLast(int arr[], int low, int high)
{
    int pivotIndex;

    if (low < high)
    {
        pivotIndex = partitionLast(arr, low, high);

        quickSortLast(arr, low, pivotIndex - 1);
        quickSortLast(arr, pivotIndex + 1, high);
    }
}

int randomIndex(int low, int high)
{
```

```
}

int partitionRandom(int arr[], int low, int high)
{
    int randomPivot;
    int pivot;
    int i;
    int j;

    randomPivot = randomIndex(low, high);

    swap(&arr[randomPivot], &arr[high]);

    pivot = arr[high];
    i = low - 1;

    for (j = low; j < high; j++)
    {
        comparisons++;
```

```
        if (arr[j] <= pivot)
        {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }

    swap(&arr[i + 1], &arr[high]);

    return i + 1;
}

void quickSortRandom(int arr[], int low, int high)
{
    int pivotIndex;

    if (low < high)


        quickSortRandom(arr, low, pivotIndex - 1);
        quickSortRandom(arr, pivotIndex + 1, high);
    }

void displayArray(int arr[], int n)

    int i;

    for (i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }

    printf("\n");

void copyArray(int source[], int destination[], int n)

    for (i = 0; i < n; i++)
    {
        destination[i] = source[i];
    }
}

void testLastPivot(char name[], int original[], int n)
{
    int arr[SIZE];

    copyArray(original, arr, n);

    comparisons = 0;
    swaps = 0;

    quickSortLast(arr, 0, n - 1);

    printf("\n%s - Last Element Pivot\n", name);
    printf("Sorted Array : ");
```

```c
void testRandomPivot(char name[], int original[], int n)
{
    int arr[SIZE];

    copyArray(original, arr, n);

    comparisons = 0;
    swaps = 0;

    quickSortRandom(arr, 0, n - 1);

    printf("\n%s - Randomized Pivot\n", name);
    printf("Sorted Array : ");
    displayArray(arr, n);
    printf("Comparisons  : %ld\n", comparisons);
    printf("Swaps        : %ld\n", swaps);
}
```

```c
    int randomData[SIZE] =
        {7, 2, 9, 1, 5, 10, 3, 8, 6, 4};
    int duplicates[SIZE] =
        {5, 3, 5, 2, 5, 8, 5, 1, 5, 4};
    printf("================================================\n");
    printf(" QUICK SORT PARTITION EFFICIENCY ANALYSIS\n");
    printf("================================================\n");
    testLastPivot("Sorted Dataset", sorted, SIZE);
    testLastPivot("Reverse Sorted Dataset", reverse, SIZE);
    testLastPivot("Random Dataset", randomData, SIZE);
    testLastPivot("Duplicate Dataset", duplicates, SIZE);
    printf("\n================================================\n");
    srand(12345);
    testRandomPivot("Sorted Dataset", sorted, SIZE);
    testRandomPivot("Reverse Sorted Dataset", reverse, SIZE);
    testRandomPivot("Random Dataset", randomData, SIZE);
    testRandomPivot("Duplicate Dataset", duplicates, SIZE);
    return 0;
}
```

Output:

```
==============================================
 QUICK SORT PARTITION EFFICIENCY ANALYSIS
==============================================

Sorted Dataset - Last Element Pivot
Sorted Array : 1 2 3 4 5 6 7 8 9 10
Comparisons  : 45
Swaps        : 0

Reverse Sorted Dataset - Last Element Pivot
Sorted Array : 1 2 3 4 5 6 7 8 9 10
Comparisons  : 45
Swaps        : 5

Random Dataset - Last Element Pivot
Sorted Array : 1 2 3 4 5 6 7 8 9 10
Comparisons  : 20
Swaps        : 8

Duplicate Dataset - Last Element Pivot
Sorted Array : 1 2 3 4 5 5 5 5 5 8
Comparisons  : 23
Swaps        : 9

==============================================
```

```
==============================================

Sorted Dataset - Randomized Pivot
Sorted Array : 1 2 3 4 5 6 7 8 9 10
Comparisons  : 24
Swaps        : 10

Reverse Sorted Dataset - Randomized Pivot
Sorted Array : 1 2 3 4 5 6 7 8 9 10
Comparisons  : 23
Swaps        : 21

Random Dataset - Randomized Pivot
Sorted Array : 1 2 3 4 5 6 7 8 9 10
Comparisons  : 19
Swaps        : 14

Duplicate Dataset - Randomized Pivot
Sorted Array : 1 2 3 4 5 5 5 5 5 8
Comparisons  : 23
Swaps        : 11
```

Conclusion

The Quick Sort partition efficiency experiment successfully measured comparisons and swaps for different input datasets.

The results demonstrate that:

- Partition quality strongly affects Quick Sort performance.
- Sorted and reverse-sorted inputs can produce poor partitions when the last element is always selected as pivot.
- Random datasets generally provide better partition behavior.
- Randomized pivot selection reduces dependence on the original ordering of the input.
- Duplicate-heavy datasets can benefit from three-way partitioning.
- Balanced partitions lead to approximately O(n log n) performance.
- Highly unbalanced partitions can lead to $O(n^2)$ performance.